TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
BỘ MÔN KHOA HỌC MÁY TÍNH & HỆ THỐNG THÔNG TIN

SỔ TAY HƯỚNG DẪN THỰC HIỆN ĐỒ ÁN 2

Đề tài: Hệ Thống AI Agent Hỗ Trợ Tra Cứu và Quản Lý Vật Tư - Thiết Bị Phòng Ban

---

CHUYÊN ĐỀ TUẦN 03
ĐÁP ÁN MẪU TUẦN 2 & BẢN CHẤT CỦA AI AGENT: KHẮC PHỤC ẢO GIÁC TÍNH TOÁN & NHẬP MÔN LANGGRAPH

Sinh viên thực hiện: Trần Huy Hoàng | **MSSV:** 20237337
Đơn vị áp dụng: Phòng Hành chính – Quản trị (Ban Quản lý Cơ sở vật chất)
Thời gian thực hiện: Tuần 3 (Kiến trúc Agent & Điều phối Đồ thị LangGraph)

---

1. Đáp Án Mẫu & Đánh Giá Chi Tiết Bài Tập Tuần 02

Trước khi bước vào kiến trúc AI Agent, thầy phân tích chi tiết đáp án của 3 bài tập Tuần 2:

1.1. Đáp án Bài tập 1: Giải thích đoạn code truy vấn ORM

```
low_stock = db.query(Asset).filter(
    Asset.category == "Vat tu tieu hao",
    Asset.quantity <= 10
).all()
```

- **Ý nghĩa kỹ thuật:** Câu lệnh này truy vấn trong bảng `assets`, lọc ra danh sách tất cả các bản ghi có phân loại là *Vật tư tiêu hao* VÀ có số lượng tồn kho ≤ 10.
- **Ý nghĩa nghiệp vụ quản trị kho:** Con số ≤ 10 đại diện cho **Ngưỡng an toàn tồn kho (Reorder Point)**. Trong thực tế, khi giấy in hay mực in chạm mức này, cán bộ phòng ban phải chủ động liên hệ nhà cung cấp đặt mua bổ sung, tránh để kho cạn kiệt đột ngột làm tê liệt hoạt động in ấn của toàn công ty.

1.2. Đáp án Bài tập 2: Kết quả chạy lệnh kiểm tra kho thực tế

Khi chạy lệnh trên Terminal, Python trả về kết quả số liệu thực tế được tính toán chuẩn xác từ CSDL SQLite:

```
{'total_items': 9, 'consumables_count': 4, 'devices_count': 5,
 'total_valuation_vnd': 86526000, 'total_valuation_str': '86,526,000 VND',
 'low_stock_items': [{'code': 'VT-MUC-2900', 'name': 'Hop muc may in Cartridge 12A'
    , 'quantity': 6, 'unit': 'Hop'}]}
```

Em thấy rằng tổng tài sản kho phòng ban là **86.526.000 VNĐ**, và hệ thống tự động phát hiện mặt hàng **Hộp mực Cartridge 12A** chỉ còn 6 hộp (cần nhập kho thêm).

1.3. Đáp án Bài tập 3: Tại sao chọn SQLite thay vì MySQL hay MongoDB?

Nếu thầy cô Hội đồng hỏi câu này, em hãy trả lời theo 3 luận điểm thuyết phục sau:

1. **Phù hợp quy mô phòng ban:** Hệ thống quản lý cho một phòng ban có khoảng vài trăm đến vài nghìn mặt hàng. SQLite là CSDL dạng tệp nhúng cục bộ (Serverless), tốc độ đọc/ghi cực nhanh, tiêu tốn ít RAM và không cần cài dịch vụ ngầm phức tạp.
2. **Tính tiện lợi và đóng gói (Zero-Configuration):** Đồ án có thể chạy ngay trên máy tính của cán bộ hoặc mang lên laptop trình chiếu trước Hội đồng mà không cần cài đặt cấu hình server MySQL, không lo lỗi cổng mạng hay mất mật khẩu root.
3. **Kiến trúc ORM linh hoạt:** Vì chúng ta sử dụng SQLAlchemy, nên nếu sau này mở rộng quy mô toàn doanh nghiệp với PostgreSQL, ta chỉ cần thay đổi 1 dòng chuỗi kết nối trong `config.py` mà không cần sửa bất kỳ dòng code nghiệp vụ nào.

2. Mục Tiêu Trọng Tâm Tuần 03: Bản Chất Của AI Agent

Trong tuần này, em sẽ bước vào phần "linh hồn" của đề tài:

1. Phân biệt rõ sự khác nhau giữa **ChatGPT thông thường** và **AI Agent**.
2. Hiểu nguyên nhân và cách khắc phục lỗi **Ảo giác tính nhẩm (Hallucination Tax)** được công bố trong bài báo khoa học *arXiv:2602.12631 (tháng 2/2026)*.
3. Làm quen với khung điều phối đồ thị trạng thái **LangGraph (StateGraph)**.

3. AI Agent Là Gì? Khác Gì So Với ChatGPT Thường?

Nhiều người lầm tưởng cài ChatGPT vào một khung chat là thành Agent. Thực tế, sự khác biệt nằm ở công thức:

$$\textbf{AI Agent} = \textbf{LLM}\ (\text{Bộ não suy luận}) + \textbf{Tools}\ (\text{Cánh tay thao tác}) + \textbf{Memory}\ (\text{Bộ nhớ}) + \textbf{Planning}\ (\text{Kế hoạch})$$

Đặc Điểm So Sánh	LLM Thông Thường (ChatGPT / Gemini)	AI Agent (Hệ Thống Của Đồ Án)
Khả năng tương tác	Chỉ biết sinh văn bản, không tương tác được với hệ thống bên ngoài.	Biết gọi các công cụ (Tools): Đọc CSDL, cào tài liệu từ internet, xuất file Word.
Tri thức thực tế	Không biết kho của công ty còn bao nhiêu ram giấy, máy chiếu phòng nào đang hỏng.	Kết nối trực tiếp với CSDL thực của phòng ban (`office_assets.db`).
Độ tin cậy số học	Dễ bị ảo giác tính nhẩm, cộng trừ tiền thường xuyên bị sai.	Không bao giờ tính nhẩm; giao toàn bộ phép toán cho code Python/SQL chạy.
Quy trình hành động	Trả lời một lượt rồi dừng lại.	Có quy trình nhiều bước: Hỏi lỗi → Hướng dẫn → Không được thì tự xuất file Word.

Bảng 1: So sánh giữa LLM thuần túy và Hệ thống AI Agent.

4. Hiện Tượng "Ảo Giác Tính Nhẩm" (Hallucination Tax)

Nghiên cứu *"AI Agents for Inventory Control: Human-LLM-OR Complementarity"* (arXiv:2602.12631, 2026) chỉ ra rằng:

- Mô hình ngôn ngữ lớn (LLM) bản chất là mô hình xác suất đoán từ tiếp theo (*Next-token prediction*), nó không có mạch tính toán logic số học trong nhân xử lý.
- Khi để LLM tự tính: *"Kho có 85 ram, xuất 15 ram, nhập thêm 30 ram, hỏi còn bao nhiêu?"*, nếu văn cảnh phức tạp hoặc số liệu lớn, LLM rất hay đoán mò ra con số sai lệch.
- **Giải pháp của đồ án (Deterministic Code Execution):** LLM chỉ làm nhiệm vụ đọc hiểu câu hỏi tiếng Việt của người dùng; khi cần tính toán, Agent tự động gọi hàm Python `commit_stock_transaction` để CPU máy tính tính toán với độ chính xác tuyệt đối 100%.

5. Nhập Môn LangGraph: Điều Phối Luồng Bằng Đồ Thị Trạng Thái

Thay vì dùng các lệnh `if...else` rối rắm, đồ án sử dụng thư viện **LangGraph** để xây dựng một đồ thị luồng xử lý:

- **State (Trạng thái):** Một đối tượng lưu giữ toàn bộ thông tin của phiên hội thoại (tin nhắn, quyền hạn người dùng, file ảnh, kết quả phản hồi).
- **Node (Nút xử lý):** Mỗi nút là một chuyên viên AI xử lý một việc cụ thể:
  - `classify_and_auth`: Phân loại câu hỏi và kiểm tra quyền RBAC.
  - `inventory_node`: Chuyên viên tra cứu kho và số liệu.
  - `support_node`: Chuyên viên hướng dẫn sửa lỗi và lập phiếu báo hỏng.
  - `enrich_node`: Chuyên viên cào tài liệu từ mạng theo model.
  - `ocr_node`: Chuyên viên đọc ảnh hóa đơn.
- **Conditional Edges (Cạnh rẽ nhánh):** Quyết định câu hỏi sẽ được chuyển đến nút nào dựa trên kết quả phân tích của nút phân loại.

```python
# Doan code khoi tao LangGraph trong src/agents/orchestrator.py
builder = StateGraph(AgentState)
builder.add_node("classify_and_auth", classify_and_auth)
builder.add_node("inventory_node", run_inventory_node)
builder.add_node("support_node", run_support_node)

builder.add_edge(START, "classify_and_auth")
builder.add_conditional_edges("classify_and_auth", route_intent)
```

6. Bài Tập Thực Hành Tuần 03 (Dành Riêng Cho Em)

Em hãy đọc kỹ tài liệu và trả lời 3 câu hỏi sau:

6.1. Bài tập 1: Giải thích cơ chế điều phối

Khi một nhân viên gõ câu: *"Máy in ở phòng hành chính bị kẹt giấy thì gỡ thế nào?"*, theo em luồng xử lý của LangGraph trong file `src/agents/orchestrator.py` sẽ kích hoạt Node nào đầu tiên và sau đó rẽ nhánh tới Node nào?

6.2. Bài tập 2: Tư duy học thuật trước Hội đồng

Em hãy giải thích bằng lời ngắn gọn: Tại sao bài báo khoa học *arXiv:2602.12631* lại coi việc AI tính nhẩm là một "khoản thuế ảo giác" (*Hallucination Tax*) đối với doanh nghiệp? Đồ án của em đã triệt tiêu khoản thuế này như thế nào?

6.3. Bài tập 3: Thực hành quan sát

Em hãy mở file `src/agents/orchestrator.py`, tìm hàm `classify_and_auth` và chỉ ra: Nếu một tài khoản quyền `user` gửi yêu cầu nhập kho bằng ảnh hóa đơn thì hệ thống sẽ xử lý ra sao?

— Hết Tài Liệu Tuần 03. Chúc em tiếp tục học tập và làm chủ kiến thức vững vàng! —